

ESP_TIMER

ESP_TIMER

- 1. ESP-Timer Overview
- 2. Core Concepts
 - 2.1 Two Dispatch Modes
 - 2.2 Timestamp Source
 - 2.3 Task Notification
- 3. Project Architecture and Flow
- 4. Hardware Dependencies
- 5. Source Code Analysis
 - 5.1 Macro Definitions and Global Variables
 - 5.2 timer_callback() — ISR Callback
 - 5.3 timer_init() — Create Timer
 - 5.4 timer_start() — Start Timer
 - 5.5 app_main() — Main Entry
- 6. Operation Flow
 - 6.1 Build and Flash
 - 6.2 Normal Output
- 7. Common Issues

1. ESP-Timer Overview

ESP-Timer is a high-precision software timer service provided by ESP-IDF, implemented based on 64-bit hardware timer for microsecond-level time measurement and periodic callbacks. Compared to FreeRTOS's `vTaskDelay()` (precision limited by tick frequency, typically ms-level), ESP-Timer can achieve **microsecond-level precision**.

Application Scenario	Typical Period	Description
Button scanning / debounce	10~20ms	Periodic GPIO reading, software filtering for debounce
LED blinking / running lights	100~500ms	Control on/off rhythm, no hardware PWM needed
Sensor periodic reading	100ms~1s	Timed sampling of temperature/humidity, light sensors, etc.
Watchdog feeding	< task timeout	Ensure system runs normally, prevent false reset
Display refresh	30~100ms	TFT/OLED page data update
Timeout detection / heartbeat	100ms~1s	Detect communication timeout, peripheral online status

This experiment creates a 10ms period **ISR mode** timer. Every 100 triggers, it prints the trigger count and total elapsed time since boot, verifying ESP-Timer's microsecond-level timing capability.

2. Core Concepts

2.1 Two Dispatch Modes

ESP-Timer callbacks can execute in two contexts:

This enum defines the two execution modes for timer callbacks: `ESP_TIMER_TASK` lets a dedicated high-priority FreeRTOS task dispatch the callback, while `ESP_TIMER_ISR` executes the callback directly in the timer's ISR context, offering lower latency.

```
typedef enum {
    // callback dispatched by dedicated high-priority FreeRTOS task
    ESP_TIMER_TASK = 0,
    // callback executed directly in timer ISR context
    ESP_TIMER_ISR = 1,
    ESP_TIMER_MAX,
} esp_timer_dispatch_t;
```

Feature	<code>ESP_TIMER_TASK</code>	<code>ESP_TIMER_ISR</code> (this experiment)
Execution Context	FreeRTOS task (not ISR)	ESP-Timer internal ISR
Latency	~tens of μ s (task scheduling overhead)	~few μ s (direct trigger)
Available API	All FreeRTOS API	Only <code>xxxFromISR()</code> version
Callback Duration Limit	Relaxed (can use <code>printf</code> and other blocking operations)	Strict (must return quickly <10 μ s)
Applicable Scenario	Complex business logic, LCD refresh	High-precision sampling, microsecond-level timing control

Purpose of selecting ISR mode in this experiment: Pursuing lowest latency, testing ESP-Timer software timer's ISR dispatch precision.

2.2 Timestamp Source

This function is the "timekeeper" of the entire experiment: calling it gives the microsecond-level time since chip boot. Regardless of what FreeRTOS is doing or whether it's interrupted, it can steadily record time.

```
// Returns microseconds since boot
int64_t esp_timer_get_time(void);
```

`esp_timer_get_time()` reads the 64-bit hardware counter with 1 μ s precision. This counter starts immediately after chip power-on, **unaffected by FreeRTOS tick or any software interrupt**, making it the most reliable time reference in ESP-IDF.

2.3 Task Notification

The simplest way for an interrupt to notify the main task is these two lines: call `vTaskNotifyGiveFromISR` in ISR to send signal, call `ulTaskNotifyTake` in main task to wait for signal—no need to pre-create semaphores or queues, zero overhead.

```
vTaskNotifyGiveFromISR(main_task_handle, NULL);

// wait for notification in main task
ulTaskNotifyTake(pdTRUE, pdMS_TO_TICKS(20));
```

Task notification is the lightest **ISR-to-Task** communication mechanism in FreeRTOS:

Communication Method	RAM Overhead	Creation API	ISR-Available Version
Task notification (this experiment)	Built-in 1 per task	No creation needed	 <code>vTaskNotifyGiveFromISR</code>
Binary semaphore	Need to pre-create SemaphoreHandle_t	<code>xSemaphoreCreateBinary()</code>	 <code>xSemaphoreGiveFromISR</code>
Queue	Need to pre-create QueueHandle_t + buffer	<code>xQueueCreate()</code>	 <code>xQueueSendFromISR</code>

This experiment's "single consumer" mode is the best use case for task notification. If multiple consumers need to wait for the same event, use semaphores.

3. Project Architecture and Flow

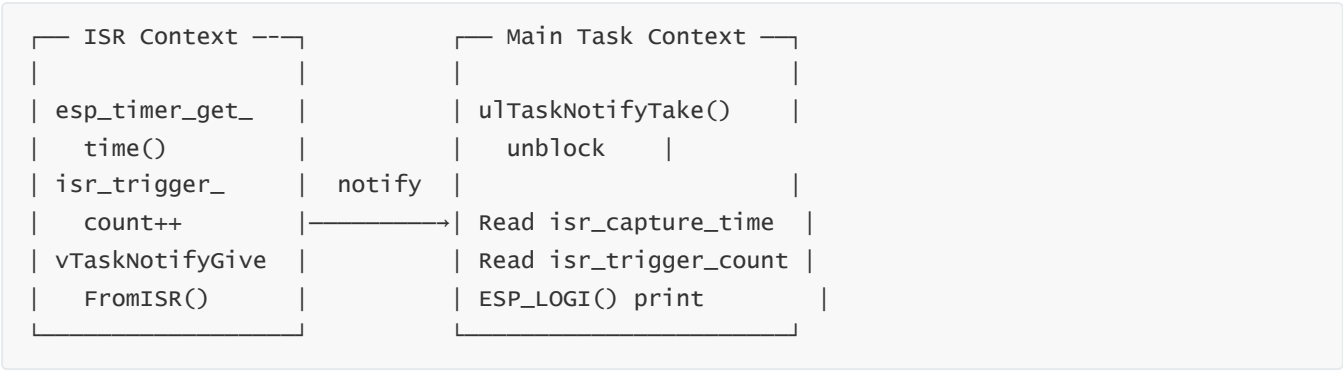
Here's the entire program's call flow: main function first prints logs, saves handle, initializes and starts timer, then enters infinite loop waiting for notification. Every 100 triggers it prints trigger count and total elapsed time since boot. Timer callback in ISR only does three things: capture timestamp, increment count, send notification.

```
app_main()
├─ ESP_LOGI()           Print "precision test"
├─ xTaskGetCurrentTaskHandle() Save main task handle (for ISR notification)
├─ timer_init()         Create ISR mode 10ms periodic timer
├─ timer_start()        Start timer
├─ while(1) Main Loop
│   └─ ulTaskNotifyTake() Block waiting for ISR notification (20ms timeout)
│       └─ Every 100 triggers:
│           └─ ESP_LOGI() Print trigger count + elapsed time

Timer callback (ISR context):
timer_callback()
├─ isr_capture_time = esp_timer_get_time() Capture timestamp
├─ isr_trigger_count++ Increment count
└─ vTaskNotifyGiveFromISR() Wake up main task
```

ISR - Main Task Data Flow:

This diagram shows the division of labor between ISR and main task: ISR only captures data, main task does calculation and printing after receiving it.



4. Hardware Dependencies

Hardware Connection: No external hardware dependency, pure software timer experiment.

Dependencies: freertos, esp_timer, esp_log

5. Source Code Analysis

5.1 Macro Definitions and Global Variables

First define some "global configuration" and "shared variables": timer period 10 milliseconds, log tag for debugging convenience, and several static variables as the core data of the entire program—timer handle, main task handle, timestamp and trigger count recorded in ISR.

```
#define TAG          "ESP_TIMER"          // Log tag
#define PERIOD_US    10000                // 10ms timer period

static esp_timer_handle_t my_timer        = NULL; // Timer handle
// Main task handle (for ISR use)
static TaskHandle_t      main_task_handle = NULL;
static volatile int64_t  isr_capture_time = 0;
static volatile int64_t  isr_trigger_count = 0;
```

`PERIOD_US = 10000` is 10ms = 100Hz trigger frequency. Reason for choosing 10ms: 100 triggers per second provides sufficient statistical sample size to evaluate timer precision (every 100 triggers = 1 second output).

5.2 timer_callback() — ISR Callback

This function is automatically called every 10 milliseconds. In the interrupt it only does three "light tasks": ① capture current time, ② trigger count +1, ③ notify main task "work arrived". Because it's running in ISR context, absolute no blocking operations (like printf).

```
static void timer_callback(void *arg)
{
    // ④ Capture microsecond timestamp
    isr_capture_time = esp_timer_get_time();
    // ⑤ Increment trigger count
    isr_trigger_count++;
    // ⑥ Wake up main task
    vTaskNotifyGiveFromISR(main_task_handle, NULL);
}
```

Why capture timestamp first, then increment count? Although the two instructions won't be preempted on single core, theoretically there could be higher priority interrupt nesting between `isr_trigger_count++` and `esp_timer_get_time()`. Capturing timestamp first ensures `isr_capture_time` is as close as possible to the actual ISR entry moment.

5.3 timer_init() — Create Timer

Creating a timer means filling the configuration struct with the previous callback function, ISR mode, timer name, etc., then calling `esp_timer_create` to register with the system. At this point the timer hasn't started yet, just prepared.

```
static void timer_init(void)
{
    esp_timer_create_args_t config = {
        // Callback function
        .callback          = timer_callback,
        // Callback argument
        .arg               = NULL,
        .name              = "demo_timer",    // Timer name
        .dispatch_method   = ESP_TIMER_ISR,
        // Don't skip unhandled events
        .skip_unhandled_events = false,
    };
    // Create timer
    ESP_ERROR_CHECK(esp_timer_create(&config, &my_timer));
    // Log creation
    ESP_LOGI(TAG, "Timer created (ISR mode, period %d ms)", PERIOD_US / 1000);
}
```

`skip_unhandled_events` parameter:

- `false` — If a callback execution time exceeds the period, system will execute backlogged callbacks as soon as possible (catch-up mechanism)
- `true` — Skip backlogged callbacks, keep only the latest

This experiment sets `false`, ensuring no trigger counts are lost.

5.4 timer_start() — Start Timer

After timer is created, it won't run yet. Need to call `esp_timer_start_periodic` with period value to formally start periodic triggering.

```
static void timer_start(void)
{
    // Start periodic timer
    ESP_ERROR_CHECK(esp_timer_start_periodic(my_timer, PERIOD_US));
    // Log start
    ESP_LOGI(TAG, "Timer started");
}
```

5.5 app_main() — Main Entry

Main function logic: ① print log marking start → ② save own task handle → ③ initialize timer → ④ record start time → ⑤ start timer → ⑥ in infinite loop, check trigger count whenever notification received, print trigger count and total microseconds since boot every 100 triggers.

```
void app_main(void)
{
    // Precision test start
    ESP_LOGI(TAG, "esp_timer precision test");

    // Save handle for ISR notification
    main_task_handle = xTaskGetCurrentTaskHandle();
    // Initialize timer
    timer_init();

    // Record program start time
    int64_t start = esp_timer_get_time();
    // Trigger count at last print
    int64_t last_print_trigger = 0;

    // Start timer
    timer_start();

    while (1) {
        // Block waiting for ISR notification
        ulTaskNotifyTake(pdTRUE, pdMS_TO_TICKS(20));
        // Read current trigger count
        int64_t trigger = isr_trigger_count;

        // Every 100 triggers print once
        if (trigger - last_print_trigger >= 100) {
            // Current timestamp
            int64_t now = isr_capture_time;
            // Total elapsed time since boot
            int64_t elapsed = now - start;

            // Update last print trigger count
        }
    }
}
```

```
        last_print_trigger = trigger;

        // Print trigger count and elapsed time
        ESP_LOGI(TAG, "[%11d] elapsed: %11d us", trigger, elapsed);
    }
}
}
```

6. Operation Flow

6.1 Build and Flash

For detailed flashing process, see: [1. Before Development](#) → [3. Build and Flash](#)

6.2 Normal Output

The following is an example of serial port output after program runs.

```
I (274) ESP_TIMER: esp_timer precision test
I (274) ESP_TIMER: Timer created (ISR mode, period 10 ms)
I (274) ESP_TIMER: Timer started
I (1284) ESP_TIMER: [100] elapsed: 1000009 us
I (2284) ESP_TIMER: [200] elapsed: 2000009 us
I (3284) ESP_TIMER: [300] elapsed: 3000009 us
I (4284) ESP_TIMER: [400] elapsed: 4000009 us
I (5284) ESP_TIMER: [500] elapsed: 5000009 us
I (6284) ESP_TIMER: [600] elapsed: 6000009 us
I (7284) ESP_TIMER: [700] elapsed: 7000009 us
I (8284) ESP_TIMER: [800] elapsed: 8000009 us
I (9284) ESP_TIMER: [900] elapsed: 9000009 us
I (10284) ESP_TIMER: [1000] elapsed: 10000009 us
```

```

I (274) ESP_TIMER: esp_timer precision test
I (274) ESP_TIMER: Timer created (ISR mode, period 10 ms)
I (274) ESP_TIMER: Timer started
I (1284) ESP_TIMER: [100] elapsed: 1000009 us
I (2284) ESP_TIMER: [200] elapsed: 2000009 us
I (3284) ESP_TIMER: [300] elapsed: 3000009 us
I (4284) ESP_TIMER: [400] elapsed: 4000009 us
I (5284) ESP_TIMER: [500] elapsed: 5000009 us
I (6284) ESP_TIMER: [600] elapsed: 6000009 us
I (7284) ESP_TIMER: [700] elapsed: 7000009 us
I (8284) ESP_TIMER: [800] elapsed: 8000009 us
I (9284) ESP_TIMER: [900] elapsed: 9000009 us
I (10284) ESP_TIMER: [1000] elapsed: 10000009 us
I (11284) ESP_TIMER: [1100] elapsed: 11000009 us
I (12284) ESP_TIMER: [1200] elapsed: 12000009 us
I (13284) ESP_TIMER: [1300] elapsed: 13000009 us
I (14284) ESP_TIMER: [1400] elapsed: 14000009 us
I (15284) ESP_TIMER: [1500] elapsed: 15000009 us
I (16284) ESP_TIMER: [1600] elapsed: 16000009 us
I (17284) ESP_TIMER: [1700] elapsed: 17000009 us
I (18284) ESP_TIMER: [1800] elapsed: 18000009 us
I (19284) ESP_TIMER: [1900] elapsed: 19000009 us
I (20284) ESP_TIMER: [2000] elapsed: 20000009 us

```

7. Common Issues

Phenomenon	Cause	Solution
<code>ulTaskNotifyTake</code> timeout no return	<code>main_task_handle</code> not saved / ISR notification failed	Confirm <code>xTaskGetCurrentTaskHandle()</code> called before <code>timer_start()</code>
<code>esp_timer_create</code> fails	Timer resources exhausted	Check for duplicate creation without delete (max depends on CONFIG)
Timer stops triggering	<code>esp_timer_stop()</code> accidentally called / timer deleted	Check for delete/stop calls in code
Deviation between timestamp acquisition and ISR execution	ISR preemption delay (other interrupts executing)	Normal phenomenon, no need to handle within $\pm 10\mu\text{s}$